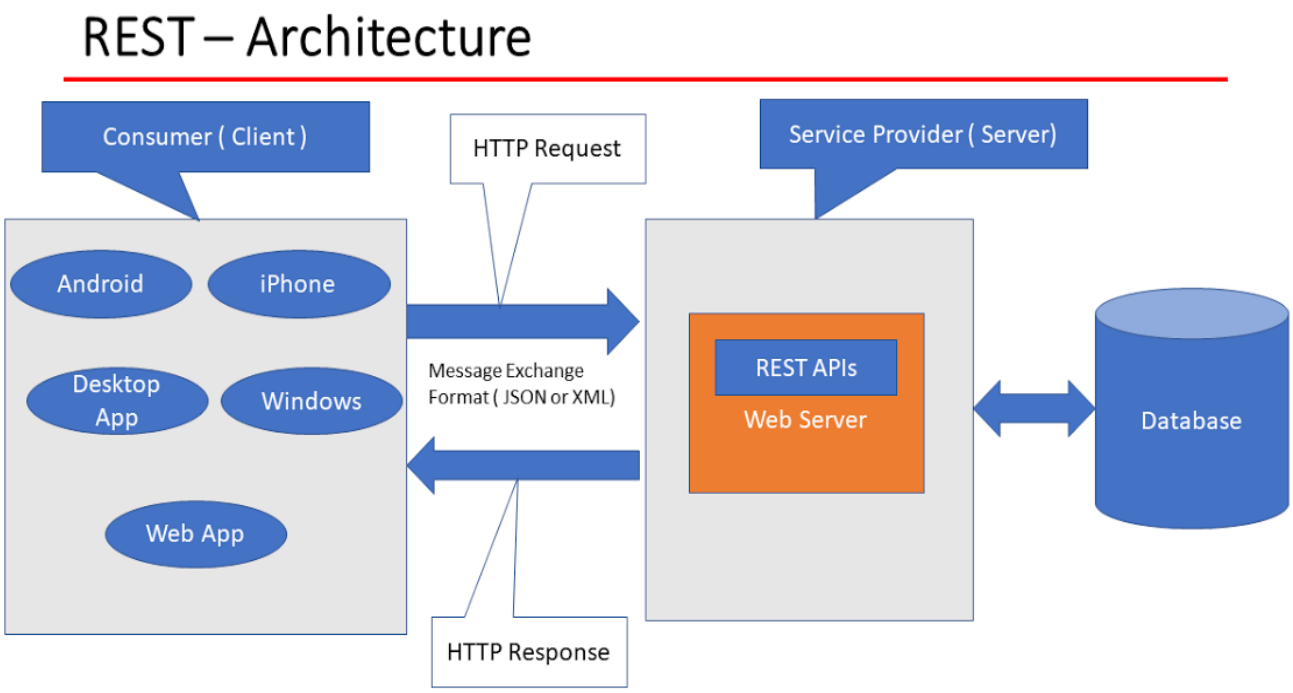
Unit 6
RESTful API

1. Introduction to RESTful API

REST (Representational State Transfer) is an architectural style used for designing network-based applications. It was introduced by Roy Fielding in 2000.

A RESTful API is an API that follows REST principles and uses HTTP protocol to enable communication between client and server.



2. Features of REST Architecture

a) Stateless – Each request is independent
b) Client-Server Architecture – Separation of frontend and backend
c) Cacheable – Responses can be stored for reuse
d) Uniform Interface – Standard way of communication
e) Layered System – Multiple layers between client and server

3. HTTP Protocol and Its Role in RESTful APIs

HTTP (HyperText Transfer Protocol):

HTTP is a communication protocol used to transfer data between client and server.

4. Working of HTTP

1. Client sends request
2. Server processes request
3. Server sends response

5. Role of HTTP in REST APIs

- REST APIs use HTTP methods to perform operations
- HTTP helps in:
  - Sending requests
  - Receiving responses
  - Transferring data
  - Providing status codes

6. HTTP Methods in RESTful APIs

HTTP methods are used to perform operations on resources.

Method	Description
GET	Retrieve data
POST	Create new data
PUT	Update existing data
DELETE	Delete data

Example:

GET /students → Get all students
POST /students → Add new student
PUT /students/1 → Update student
DELETE /students/1 → Delete student

Real-Life Example

In an online shopping system:

- GET → View products
- POST → Place order
- PUT → Update address
- DELETE → Cancel order

7. URLs and Resource Representation

Resource: A resource is any data such as student, product, or user.

PrinceXCEL

URL (Uniform Resource Locator)
URL is used to identify resources.

Examples

/students
/students/1
/products/10

- /students → All students
- /students/1 → Specific student

8. Resource Representation

Data is transferred in formats like:

- JSON (most common)
- XML

Example (JSON)

```
{
    "id": 1,
    "name": "Prince",
    "course": "CSE"
}
```

9. Best Practices for RESTful API Design

1. Use Proper HTTP Methods

- GET → Retrieve data
- POST → Create data
- PUT → Update data
- DELETE → Remove data

2. Use Meaningful URLs

✓ Good:
/students
/orders/5

3. Use HTTP Status Codes

Code	Meaning
200	OK
201	Created
400	Bad Request
404	Not Found
500	Server Error

PrinceXCEL

4. Use JSON Format

- Lightweight
- Easy to understand

5. Keep API Stateless

- Each request must contain all information
- Server should not store client state

6. Use Proper Naming Convention

- Use nouns instead of verbs

  ✓ /students

  ❌ /getStudents

10. Advantages of RESTful APIs

- Simple and easy to use
- Scalable
- Fast performance
- Platform independent

11. Disadvantages

- Stateless (no memory)
- Security issues
- May require multiple requests

12. API Versioning

Definition

API Versioning is the process of managing changes in an API without affecting existing users. It allows developers to introduce new features while maintaining compatibility with older versions.

Need for API Versioning

- To avoid breaking existing client applications
- To support multiple versions simultaneously
- To allow gradual migration to newer versions
- To maintain backward compatibility

Types of API Versioning

1. URI Versioning (Most Common)

Version is included in the URL.

PrinceXCEL

Example:

/api/v1/students
/api/v2/students

Header Versioning

Version is specified in HTTP headers.

Example:

Accept: application/vnd.api.v1+json

2. Query Parameter Versioning

Version is passed in query string.

Example:

/students?version=1

13. Error Handling in RESTful APIs

Definition

Error handling is the process of identifying, managing, and returning meaningful error responses to the client when something goes wrong.

Importance

- Helps client understand the issue
- Improves debugging
- Enhances user experience

14. Common HTTP Status Codes

Code	Meaning
200	OK (Success)
201	Created
400	Bad Request
401	Unauthorized
403	Forbidden
404	Not Found
500	Internal Server Error

PrinceXCEL

Standard Error Response Format

```
{
    "error": "Student not found",
    "status": 404,
    "message": "No student exists with given ID"
}
```

15. Consuming RESTful APIs

Definition

Consuming an API means using it to send requests and receive responses from a server.

1. Using cURL

What is cURL?

cURL is a command-line tool used to send HTTP requests.

Uses

- Testing APIs
- Debugging
- Automation

2. Using Postman

What is Postman?

Postman is a GUI-based tool used for testing APIs.

Features

- Easy to use interface
- Supports all HTTP methods
- Shows response in structured format

Steps to Use

1. Select method (GET/POST)
2. Enter API URL
3. Click Send
4. View response

16. Using Python Requests Library

What is Requests Library?

A Python library used to send HTTP requests easily.

PrinceXCEL

Example

```
import requests
response = requests.get("https://api.example.com/students")
print(response.json())
```

Advantages

- Simple syntax
- Easy integration in applications
- Supports all HTTP methods

17. Building RESTful APIs using Frameworks

Using Flask

Definition

Flask is a lightweight Python web framework used to build REST APIs.

Example

```
from flask import Flask, jsonify

app = Flask(__name__)
@app.route('/students', methods=['GET'])
def get_students():
return jsonify({"students": ["A", "B"]})

app.run()
```

Advantages

- Simple and easy
- Flexible
- Good for beginners

Using FastAPI

Definition

FastAPI is a modern, high-performance Python framework for building APIs.

Example

```
from fastapi import FastAPI

app = FastAPI()

@app.get("/students")
def get_students():
    return {"students": ["A", "B"]}
```

PrinceXCEL

Advantages

- Very fast
- Automatic API documentation
- Supports async programming

Flask vs FastAPI

Feature	Flask	FastAPI
Speed	Moderate	Very Fast
Learning	Easy	Moderate
Documentation	Manual	Automatic

18. Building Scalable RESTful APIs

Definition

Scalability is the ability of a system to handle increased load efficiently.

Techniques for Scalability

- Load balancing
- Caching responses
- Database optimization
- Microservices architecture

Securing RESTful APIs

Need for Security

- Protect user data
- Prevent unauthorized access

Common Security Methods

- Authentication (Login system)
- Authorization (Access control)
- HTTPS (Secure communication)
- API keys and tokens

*****

PrinceXCEL

Important Questions

1. Define RESTful API.
2. Explain REST Architecture Principles.
3. Explain HTTP Methods with Example.
4. What is Resource and URL in REST API?
5. Explain HTTP Status Codes.
6. Explain API Versioning.
7. Explain Error Handling in REST API.
8. Explain API Consumption Tools
9. Explain Flask and FastAPI
10. Write Advantages of REST API
11. What is Scalability in REST API?
12. Explain Security in REST APIs

PrinceXCEL